

SIGCB-QR

Sistema Integral de Gestión de Biblioteca con Códigos QR

Informe técnico — Entrega Final

Equipo SIGCB-QR

Universidad Técnica Estatal de Quevedo (UTEQ)

1 de septiembre de 2026

Versión 1.0.0-rc

Resumen

Se presenta SIGCB-QR, un sistema de gestión bibliotecaria universitaria construido como API REST en Spring Boot con un frontend Laravel que actúa como *Backend for Frontend*. El informe documenta la arquitectura y sus decisiones, y sobre todo el **estudio empírico** realizado sobre el sistema en marcha: una auditoría de seguridad de 42 comprobaciones basada en OWASP API Security Top 10 [22], la medición de cobertura con JaCoCo, cuatro corridas de carga con k6 y una auditoría de frontend con Lighthouse.

El resultado principal no son las cifras favorables, sino los **tres defectos** que las auditorías descubrieron en un sistema cuya suite de pruebas se daba por correcta: el endpoint del catálogo devolvía 500 en todo acierto de caché, toda ruta inexistente devolvía 500 filtrando mensajes internos del framework, y la política de seguridad de contenido bloqueaba los iconos, la tipografía y el módulo de códigos QR de la propia aplicación. Ninguno era detectable midiendo cobertura. Se analiza por qué, se declaran las amenazas a la validez de cada medición y se documenta un caso en que el propio instrumento de medida produjo un resultado falso.

Palabras clave: gestión bibliotecaria, arquitectura de software, ADR, OWASP, evidencia empírica, reproducibilidad, validez.

Índice

1. Introducción	3
1.1. Contexto y problema	3
1.2. Objetivos	3
1.3. Estructura del informe	3
2. Arquitectura	3
2.1. Visión general	3
2.2. Por qué dos pilas: el patrón Backend for Frontend	3
2.3. Decisiones registradas	4
2.4. Seguridad	4
2.5. Acceso a datos	5
3. Método empírico	5
3.1. Diseño	5
3.2. Reglas de reporte	5
3.3. Entorno	5
3.4. Reproducibilidad	5

4. Bloque 1: auditoría de seguridad	6
4.1. Instrumento	6
4.2. Resultados	6
4.3. Limitaciones declaradas	6
5. Bloque 2: pruebas y cobertura	6
5.1. Estado de partida	6
5.2. Resultados tras la corrección	7
5.3. Interpretación	7
6. Bloque 3: rendimiento	7
6.1. Un defecto del instrumento, corregido antes de medir	7
6.2. Resultados	7
6.3. Un resultado contrario a la hipótesis	8
7. Bloque 4: calidad del frontend	8
7.1. Por qué el rendimiento <i>bajó</i> al corregir un defecto	8
8. Defectos encontrados	8
8.1. D1 — El acierto de caché devolvía 500	9
8.2. D2 — Toda ruta inexistente devolvía 500	9
8.3. D3 — La CSP bloqueaba los recursos de la propia aplicación	9
8.4. Lectura conjunta	9
9. Amenazas a la validez	10
9.1. Validez de constructo	10
9.2. Validez interna	10
9.3. Validez externa	10
9.4. Validez de conclusión	11
10.Ética y tratamiento de datos	11
11.Conclusiones	11
11.1. Trabajo futuro	12

1. Introducción

1.1. Contexto y problema

La gestión manual del préstamo bibliotecario en la UTEQ obliga a registrar en papel o en hojas de cálculo cada entrega y devolución de ejemplares, lo que dificulta conocer la disponibilidad real del acervo, aplicar el reglamento de multas de forma consistente y obtener indicadores de uso.

SIGCB-QR automatiza autenticación, catálogo, inventario, préstamos, devoluciones, reservas, multas, códigos QR, auditoría y reportes.

1.2. Objetivos

- O1.** Construir un sistema web funcional que cubra el ciclo completo del préstamo bibliotecario.
- O2.** Aplicar ingeniería de requisitos conforme a ISO/IEC/IEEE 29148:2018 [1], con trazabilidad verificable entre requisito, código, prueba y evidencia.
- O3.** Documentar las decisiones de arquitectura con sus alternativas y su coste, siguiendo la práctica de los ADR [20].
- O4.** Producir **evidencia empírica reproducible** de seguridad, cobertura, rendimiento y calidad del frontend.
- O5.** Garantizar la reproducibilidad del entorno de medición [6].

1.3. Estructura del informe

La sección 2 describe la arquitectura y sus decisiones. La sección 3 expone el método empírico. Las secciones 4 a 7 presentan los cuatro bloques de medición. La sección 8 analiza los tres defectos encontrados. La sección 9 declara las amenazas a la validez, y la 11, las conclusiones.

2. Arquitectura

2.1. Visión general

El sistema consta de dos aplicaciones de servidor separadas por una frontera HTTP:

```
Navegador --HTTP--> Laravel 13 (BFF) --HTTP/JWT--> Spring Boot 3.5 --> PostgreSQL 16
Blade + Alpine JPA + procedimientos Redis 7
```

2.2. Por qué dos pilas: el patrón Backend for Frontend

La convivencia de Java y PHP en un mismo repositorio requiere justificación explícita; sin ella parece incoherencia y no diseño. La decisión completa está en **ADR-0002**; se resume aquí.

Laravel no contiene lógica de negocio: renderiza vistas y delega todo acceso a datos en la API. Es el patrón *Backend for Frontend* [16], y se sostiene sobre tres reglas que pueden auditarse en el repositorio, no sobre buenas intenciones:

- 1. **Laravel no habla con PostgreSQL.** No hay modelos Eloquent de dominio ni migraciones de negocio; las únicas tablas que conoce son las de su propia infraestructura.
- 2. **Todo acceso a datos pasa por ApiClient**, un único punto de salida HTTP.

3. **Ninguna regla de negocio se duplica.** La interfaz puede ocultar un botón por comodidad, pero la decisión que cuenta es la del servidor: un estudiante que llame directamente al endpoint recibe 403, y así se verifica empíricamente en la sección 4.

Es una aplicación del principio de ocultación de información de Parnas [23]: el frontend desconoce el modelo de datos, de modo que puede sustituirse sin tocar el dominio.

El coste es real y se declara: dos entornos de ejecución que mantener, un salto de red adicional por petición y el riesgo permanente de que la lógica se filtre hacia el BFF.

2.3. Decisiones registradas

ADR	Decisión	Coste asumido
0001	Registrar las decisiones como ADR	15–30 min por decisión
0002	Laravel como BFF sobre la API Spring Boot	Dos pilas; salto de red extra
0003	JWT en cookie <code>HttpOnly</code> + sesión del BFF	Superficie de CSRF
0004	Errores como <code>ProblemDetail</code> (RFC 7807)	Dos formas de cuerpo en la API
0005	CRUD por ORM; agregaciones por procedimientos	Lógica en dos lenguajes
0006	Cachear sólo tipos con ida y vuelta demostrada	Una prueba por tipo cacheado
0007	Anclar imágenes por digest SHA256	Las actualizaciones dejan de llegar solas
0008	Flyway como fuente única del esquema	Corregir exige otra migración
0009	Revocar JWT por lista negra de <code>jti</code>	Una consulta por petición

Cuadro 1: Decisiones de arquitectura y su coste. Detalle en `docs/adr/`.

Cada ADR declara consecuencias negativas de forma obligatoria: una decisión sin coste probablemente no era una decisión.

2.4. Seguridad

- **Autenticación.** JWT firmado con HS384 [12], con los claims `iss`, `aud`, `nbf`, `iat` y `jti`. Se transporta en cookie `HttpOnly` con `SameSite=Strict` [5], de modo que no es accesible desde JavaScript (**ADR-0003**).
- **Revocación.** La lista negra de `jti` en PostgreSQL hace efectivo el cierre de sesión. Se eligió la base de datos y no Redis porque una decisión de seguridad no debe depender de un componente cuyo contrato es poder perder datos (**ADR-0009**).
- **Contraseñas.** bcrypt con coste 10 y salt único por usuario [24].
- **Autorización.** Por rol con `@PreAuthorize`, aplicada en el servidor.
- **Errores.** `application/problem+json` conforme a RFC 7807 [18], hoy obsoleta por la RFC 9457 [19].

2.5. Acceso a datos

Reparto según la naturaleza de la operación (**ADR-0005**): ORM para el CRUD elemental; trece procedimientos y funciones almacenados para las agregaciones y para las operaciones de varios pasos que deben ser atómicas. El esquema lo gobierna Flyway como fuente única [2], con `ddl-auto: validate` para que una divergencia entre entidad y tabla impida el arranque (**ADR-0008**).

3. Método empírico

3.1. Diseño

El estudio es un **caso único con mediciones instrumentadas** [25], no un experimento controlado: no hay grupo de comparación ni asignación aleatoria. Las afirmaciones son descriptivas y no causales, salvo donde se indique lo contrario y se justifique.

3.2. Reglas de reporte

Se aplican tres reglas, declaradas en `ETHICS.md` y adoptadas a raíz de observaciones formales recibidas en entregas anteriores:

1. **Ninguna cifra se publica sin la orden que la produjo**, con fecha, entorno y salida cruda [26].
2. **Lo no medido se marca como no medido**. Una casilla «pendiente» es una respuesta legítima; un número plausible sin medición no lo es.
3. **Los hallazgos negativos se publican**, incluidos los defectos del propio instrumento.

3.3. Entorno

Componente	Versión
Anfitrión	Windows 11 Pro 26200; Docker 29.5.2 sobre WSL2
API	Java 21 (Temurin), Spring Boot 3.5.16
Frontend	PHP 8.3, Laravel 13.8, Apache 2.4.68
Base de datos	PostgreSQL 16.15 (anclada por digest)
Caché	Redis 7 (anclada por digest)
Instrumentos	curl 8.19.0; k6 (grafana/k6); Lighthouse 12.8.2; JaCoCo 0.8.12
Datos	Semilla sintética: 20 libros, 69 ejemplares, 6 usuarios

Cuadro 2: Entorno de medición. Las imágenes están ancladas por digest SHA256 (**ADR-0007**), de modo que las corridas son comparables entre sí.

3.4. Reproducibilidad

```
make up # levanta el sistema con imagenes ancladas por digest
make verify # valida digests, matriz de trazabilidad e indice de ADR
make test # 41 pruebas con cobertura
make audit # 42 comprobaciones de seguridad OWASP
```

4. Bloque 1: auditoría de seguridad

4.1. Instrumento

`scripts/owasp-audit.sh` ejecuta 42 comprobaciones sobre el sistema en marcha, cada una una petición HTTP real cuyo código de estado o cabecera se compara con el valor esperado. Cubre los bloques API1, API2, API3, API5, API8 y API4 de OWASP API Security Top 10 [22] y A03 y A05 de OWASP Top 10 [21].

4.2. Resultados

Bloque	N	Resultado
API1 — Autorización a nivel de objeto	4	Superado
API2 — Autenticación rota	9	Superado
API3 — Exposición excesiva de propiedades	2	Superado
API5 — Autorización a nivel de función	4	Superado
API8 / A05 — Configuración incorrecta	16	Superado
A03 — Inyección	6	Superado
API4 — Consumo sin restricción	1	Superado
Total	42	42 superadas, 0 fallidas

Cuadro 3: Auditoría OWASP. Salida cruda en `docs/mediciones/seguridad/owasp-audit-raw.txt`.

Tres verificaciones merecen mención por su diseño:

- **Comprobación de control.** Junto a cada 403 esperado se verifica que la misma petición con credenciales de administrador devuelve 200. Sin ella, un 403 constante por cualquier motivo —un servicio caído— pasaría por seguridad correcta.
- **Cuerpo válido en la prueba de BFLA.** Con un cuerpo vacío la API responde 400, porque la validación se ejecuta antes que `@PreAuthorize`; una auditoría que enviara `{}` no habría probado nada sobre la autorización.
- **Verificación del efecto, no sólo del código.** Tras el intento de creación no autorizada se comprueba que *no se creó ningún usuario*. Un 403 sería inútil si el recurso se hubiese creado igualmente.

4.3. Limitaciones declaradas

No es una prueba de penetración: sin *fuzzing*, sin inyección ciega o temporal, sin análisis de dependencias y sin escáner automatizado. Todo se midió sobre HTTP local, de modo que `Secure`, `HSTS` y `TLS` quedan sin verificar. La CSP admite `'unsafe-inline'` [27], lo que limita su eficacia real frente a XSS. El límite de tasa es en memoria y por instancia: con varias réplicas, el límite efectivo se multiplica.

5. Bloque 2: pruebas y cobertura

5.1. Estado de partida

Al iniciar esta entrega, la suite **no estaba en verde**, pese a lo declarado en la entrega anterior: `mvn verify` terminaba en `BUILD FAILURE` con 4 errores sobre 36 pruebas. Dos venían

de que `AuthController` pasó a depender de `RateLimitService` sin actualizar el doble del test, y dos de que las pruebas del filtro JWT simulaban un método que el filtro ya no llamaba –es decir, no ejercitaban el camino real [14]. Ambos casos son cambios en producción cuyas pruebas no se volvieron a ejecutar, que es justamente lo que la integración continua existe para impedir [10].

5.2. Resultados tras la corrección

Contador	Cubierto	Total	%
Instrucciones	1 530	4 728	32,36
Ramas	34	254	13,39
Líneas	429	1 099	39,04
Complejidad ciclomática	92	391	23,53
Métodos	83	264	31,44

Cuadro 4: Cobertura JaCoCo sobre 41 pruebas, 0 fallos. Umbral configurado: 30 % de líneas.

5.3. Interpretación

El reparto importa más que el total: `security` (45,5 %) y `service` (41,2 %) concentran las reglas cuya rotura tendría consecuencias, mientras que `model.entity` (1,5 %) son entidades con Lombok cuya cobertura carece de significado –probar un *getter* generado no aporta información.

El dato preocupante no es el 39 % de líneas sino el **13,39 % de ramas**: las pruebas recorren el camino feliz y dejan casi todas las bifurcaciones sin ejercitar. Es coherente con el resultado de la sección 8 y con la evidencia de que la cobertura no está fuertemente correlacionada con la efectividad de una suite [11]: mide qué líneas se ejecutan, no si el sistema hace lo que debe [29].

6. Bloque 3: rendimiento

6.1. Un defecto del instrumento, corregido antes de medir

La versión previa del arnés de k6 iniciaba sesión *en cada iteración de cada usuario virtual*. Con el límite de 5 intentos por IP y minuto, a partir del quinto login todo eran respuestas 429, que se sirven sin tocar la base de datos. El resultado habría sido inválido en dos direcciones: la tasa de error se dispara y **el p95 sale artificialmente bajo**. La prueba habría medido la velocidad del limitador de tasa presentándola como velocidad del catálogo. El inicio de sesión se movió a `setup()`, fuera de la ventana de medición.

6.2. Resultados

Carga: 0→5 usuarios virtuales (10 s), 5→20 (20 s), 20→0 (10 s); ≈370 iteraciones por corrida sobre `GET /api/libros`.

#	Caché	JVM	p95	media	mediana	errores
1	fría	fría	83,43 ms	57,19 ms	17,28 ms	0 %
2	caliente	templada	73,02 ms	28,21 ms	20,48 ms	0 %
3	caliente	caliente	64,89 ms	25,61 ms	17,79 ms	0 %
4	fría	caliente	26,41 ms	12,66 ms	9,40 ms	0 %

Cuadro 5: Corridas de k6, en orden cronológico real. Umbral REQ-NF-001: $p95 \leq 200$ ms. Salidas crudas en `docs/mediciones/perf/`.

Todas cumplen el umbral con holgura, y ninguna registró un solo error en 1 474 peticiones. El hit ratio de Redis fue del 99,73 % (376 aciertos, 1 fallo).

6.3. Un resultado contrario a la hipótesis

La corrida 4 tiene la caché **vacía** y es la más rápida de todas: su p95 es menos de la mitad que el de las corridas con caché llena. La conclusión es incómoda pero es la que sostienen los datos:

A esta escala, el p95 lo determina el calentamiento de la JVM, no el estado de la caché de Redis.

La explicación es sencilla y comprobable: la semilla tiene 20 libros. Una página de diez filas se resuelve en PostgreSQL con un índice y un LIMIT, y ese trabajo es comparable –o menor– al de serializar el resultado y hacer un viaje de ida y vuelta a Redis. Lo caro al principio es que el JIT compile las rutas de Hibernate, Jackson y Spring Security, tal como advierte la literatura sobre medición rigurosa en Java [9].

Por tanto, **no puede afirmarse que la caché mejore el rendimiento de este sistema**. Sigue justificada por **ADR-0006** como mecanismo que escalará con un acervo real de decenas de miles de títulos, pero eso es una previsión, no una medición. Afirmar lo contrario sería exactamente el tipo de conclusión que estas reglas de reporte pretenden evitar.

7. Bloque 4: calidad del frontend

Lighthouse 12.8.2 sobre /login, factor de forma móvil con estrangulamiento simulado.

Categoría	CSP rota	CSP corregida
Rendimiento	100	82
Accesibilidad	100	100
Buenas prácticas	93	100
SEO	91	91

Cuadro 6: Lighthouse antes y después de corregir la política de seguridad de contenido. La cifra válida es la de la derecha.

7.1. Por qué el rendimiento *bajó* al corregir un defecto

El 100 de la primera corrida no era mérito de la aplicación: era el efecto de que el navegador estuviera bloqueando sus propias hojas de estilo y tipografías. Una página que no llega a cargar Font Awesome ni Poppins pinta antes. Al corregir la CSP, la página hace lo que siempre debió hacer –descargar dos hojas de estilo y seis archivos de tipografía desde tres dominios externos– y el First Contentful Paint pasa de 1,4s a 3,6s.

Es decir: **la primera corrida midió una página rota y la premió por ello**. Una métrica de rendimiento aislada, sin comprobar que la página funciona, puede recompensar el defecto que debería denunciar. El 82 es la línea base honesta.

8. Defectos encontrados

Las auditorías descubrieron tres defectos reales en un sistema cuya suite estaba, en apariencia, en verde. Ninguno era detectable midiendo cobertura.

8.1. D1 — El acierto de caché devolvía 500

El método cacheado devolvía `Page/PageImpl`. El serializador JSON de Redis *escribe* ese objeto sin problema pero *no puede volver a leerlo*: `PageImpl` carece de constructor sin argumentos.

```
$ curl -o /dev/null -w '%{http_code}\n' -b cookie.txt localhost:8080/api/libros
200 # fallo de cache: se sirve de la base y se escribe en Redis
$ curl -o /dev/null -w '%{http_code}\n' -b cookie.txt localhost:8080/api/libros
500 # acierto de cache: no se puede leer lo escrito
```

El endpoint principal del catálogo fallaba **en todo acierto de caché**, justo el caso que la caché existía para acelerar. Como la primera petición tras cada expiración del TTL funcionaba, el fallo parecía intermitente.

Por qué no lo vio la cobertura. El método figuraba como cubierto: las pruebas lo llamaban con Mockito, sin pasar por el proxy de `@Cacheable`. La línea estaba cubierta; el comportamiento, no.

Corrección. El método devuelve `PageResponse<LibroResponse>`, un DTO con ida y vuelta demostrada por la prueba `CacheSerializationTest`, que además fija la causa raíz: si `PageImpl` dejara de fallar, el motivo del **ADR-0006** habría cambiado. Se descubrió de paso un segundo defecto latente: la clave de caché ignoraba el criterio de orden, de modo que dos peticiones con distinto `sort` compartían entrada.

8.2. D2 — Toda ruta inexistente devolvía 500

`NoResourceFoundException` no tenía manejador propio y caía en el manejador genérico, que además construía el detalle concatenando el mensaje de la excepción:

```
{"status":500,"detail":"Error interno del servidor: No static resource actuador/env."}
```

Dos consecuencias: una URL mal escrita era indistinguible de un fallo real del servidor, lo que contradice el propósito de la RFC 7807 [18]; y el manejador devolvía sin filtrar el mensaje de cualquier excepción no controlada a cualquier cliente —el mismo camino por el que se expusieron trazas internas durante el diagnóstico de D1.

Corrección. Manejadores propios para 404 y 405, y detalle genérico en el manejador de último recurso, con la traza registrada en el servidor. La prueba de regresión envía una excepción cuyo mensaje contiene una cadena de conexión con credenciales y verifica que no aparece en la respuesta.

8.3. D3 — La CSP bloqueaba los recursos de la propia aplicación

La política sólo admitía `'self'`, mientras que ambos *layouts* cargan Font Awesome y la tipografía Poppins desde CDN, y el módulo de códigos QR carga su biblioteca desde `cdnjs`. En el navegador no se dibujaba ningún icono, la tipografía caía a la de reserva y el módulo QR quedaba inoperativo, en *todas* las páginas.

El fallo era silencioso: el servidor respondía 200 y el HTML era correcto. Sólo la consola del navegador lo delataba, por lo que ni las pruebas ni la auditoría con `curl` podían detectarlo. Hizo falta un navegador real.

8.4. Lectura conjunta

Los tres defectos comparten un patrón: **se manifiestan en la integración entre componentes**, no dentro de una unidad. Caché y serializador; framework web y manejador de excepciones; navegador y política de seguridad. Una suite compuesta casi enteramente de pruebas unitarias con dobles —como la de este proyecto, según la sección 5— es ciega por construcción ante esa clase de fallo. Es la explicación concreta, en este sistema, de por qué la cobertura no predice la efectividad de una suite [11].

9. Amenazas a la validez

Se sigue la clasificación de Wohlin et al. [28], basada en Cook y Campbell [8].

9.1. Validez de constructo

¿Miden los instrumentos lo que dicen medir?

- **La cobertura no mide calidad.** Mide qué líneas ejecutan las pruebas. Los tres defectos de la sección 8 ocurrieron en código contabilizado como cubierto [11].
- **42 comprobaciones superadas no significan «sistema seguro».** Significa que 42 controles concretos se comportan como se espera. La ausencia de evidencia de vulnerabilidad no es evidencia de ausencia.
- **El SUS mide percepción, no eficacia** [7]. Un sistema puede puntuar alto y aun así hacer que la gente tarde el doble.
- **La puntuación de Lighthouse puede premiar un defecto,** como quedó demostrado en la sección 7.

9.2. Validez interna

¿Están las conclusiones causalmente justificadas?

- **El orden de las corridas se confunde con el calentamiento de la JVM.** Las corridas no se aleatorizaron, así que el efecto de la caché y el del calentamiento están confundidos [9]. Por eso la sección 6 se abstiene de atribuir la mejora a la caché.
- **Cliente y sistema comparten máquina.** La carga del generador compete por CPU con lo medido, un sesgo de medición conocido [15].
- **Sin repeticiones suficientes.** Tres corridas comparables no permiten un intervalo de confianza defendible; las cifras son descriptivas.
- **El instrumento produjo un resultado falso.** La primera corrida de la auditoría marcó tres 401 donde esperaba 403, y el fallo estaba en el propio script: creaba la sesión del bloque de cierre de sesión *copiando* el archivo de cookies del estudiante, de modo que ambos compartían token y la revocación por `jti` invalidaba también la sesión del estudiante. La revocación funcionaba tan bien que rompió la auditoría. Interpretar aquellos 401 como fallo de autorización habría llevado a «arreglar» algo que no estaba roto.

9.3. Validez externa

¿Hasta dónde se generalizan los resultados?

- **El volumen de datos no es representativo:** 20 libros y 69 ejemplares, frente a los varios órdenes de magnitud de una biblioteca universitaria real. Es la limitación más severa, y afecta directamente a la conclusión sobre la caché [13].
- **Despliegue de una sola máquina,** sin latencia de red entre componentes ni réplicas.
- **20 usuarios virtuales** es carga baja: no se localizó el punto de saturación.
- **Un solo endpoint medido.** Préstamos y reportes, que ejecutan procedimientos almacenados, tienen otro perfil y no se midieron.
- **Una sola página auditada con Lighthouse,** y sin sesión.

9.4. Validez de conclusión

- **Sin análisis estadístico inferencial.** No hay contraste de hipótesis ni tamaño del efecto; los percentiles son descriptivos.
- **El SUS no tiene datos.** Ninguna afirmación sobre usabilidad percibida está respaldada, y así se declara en lugar de rellenarlo.
- **Los autores son los evaluadores.** Quien construyó el sistema diseñó las comprobaciones, eligió los umbrales y decidió qué medir. Es la amenaza más difícil de mitigar, y el protocolo del SUS (sección 7 y docs/mediciones/usabilidad/) la reconoce explícitamente como sesgo de cortesía [17]. La contramedida aplicada es hacer cada medición reproducible por un tercero con una sola orden.

10. Ética y tratamiento de datos

El conjunto de datos es sintético: no hay datos de personas reales. Aun así, el sistema está diseñado bajo el supuesto de que el **historial de préstamos es un dato sensible**: lo que una persona lee permite inferir su ideología, su estado de salud o sus creencias, principio largamente sostenido por la tradición bibliotecaria [3]. Se aplican minimización, limitación de la finalidad y confidencialidad por defecto.

ETHICS.md declara también lo que el sistema *no* protege todavía: no hay cifrado en reposo del historial, no existe flujo de acceso ni de supresión a petición del titular, y no hay política de retención. Declarar las carencias forma parte de la responsabilidad profesional [4].

Se usaron asistentes de programación basados en modelos de lenguaje como apoyo a la documentación, la generación de pruebas y la revisión de código. Todo artefacto generado fue ejecutado y verificado antes de incorporarse; la responsabilidad sobre lo entregado es del equipo.

11. Conclusiones

- C1. El sistema cumple sus requisitos medibles.** 42/42 comprobaciones de seguridad, p95 entre 26 y 83 ms frente a un umbral de 200 ms, 0 % de error en 1 474 peticiones, 41 pruebas en verde y accesibilidad de 100 en las comprobaciones automáticas.
- C2. Auditar el sistema en marcha encontró lo que las pruebas no.** Tres defectos reales – uno de ellos rompía el endpoint principal en todo acierto de caché– aparecieron al ejercitar el sistema con curl, con k6 y con un navegador. Los tres viven en las costuras entre componentes, donde una suite de pruebas unitarias con dobles es ciega por construcción.
- C3. Una métrica puede premiar un defecto.** Lighthouse dio 100 en rendimiento a una página cuyos recursos el navegador bloqueaba, y el arnés de carga original habría dado un p95 excelente midiendo el limitador de tasa. Una métrica sin comprobación de que el sistema funciona no es evidencia.
- C4. El instrumento tiene sus propias amenazas a la validez.** El script de auditoría produjo tres falsos positivos por un defecto propio. Se documenta porque un instrumento que no se audita a sí mismo es tan poco fiable como el sistema que mide.
- C5. La trazabilidad debe ser ejecutable.** Una matriz que declaraba como verificadas cuatro clases de prueba inexistentes se corrigió, pero lo que impide que vuelva a ocurrir no es la corrección: es que `validate-traceability.sh` extraiga el inventario real de pruebas del repositorio y falle en CI si la matriz cita algo que no existe. Lo mismo vale para los digest de imagen, el índice de ADR y el diccionario de datos, todos verificados automáticamente.

C6. No puede afirmarse que la caché mejore el rendimiento a esta escala. Los datos sugieren lo contrario, y la conclusión se declara aunque contradiga la motivación de haberla implementado.

11.1. Trabajo futuro

1. Pruebas de integración con caché activa y con base real, que habrían detectado D1.
2. Pruebas de rama en los servicios, hoy al 13,39 %.
3. Administrar el SUS a los 12 participantes previstos.
4. Repetir las mediciones de rendimiento con un acervo de decenas de miles de títulos, único modo de contrastar la hipótesis sobre la caché.
5. Alojar tipografías, iconos y `qrcode.js` en el propio contenedor, para volver a `'self'` en toda la CSP y quitar tres dominios de la ruta crítica.
6. Implementar acceso, portabilidad y supresión de datos personales, y una política de retención.

Referencias

- [1] ISO/IEC/IEEE 29148:2018 — Systems and software engineering — Life cycle processes — Requirements engineering, 2018.
- [2] Scott W. Ambler and Pramod J. Sadalage. *Refactoring Databases: Evolutionary Database Design*. Addison-Wesley, 2006.
- [3] American Library Association. Privacy: An interpretation of the library bill of rights, 2019.
- [4] Association for Computing Machinery. ACM code of ethics and professional conduct, 2018.
- [5] Steven Bingler, Mike West, and John Wilander. Cookies: HTTP state management mechanism. Technical report, Internet Engineering Task Force, 2024. Internet-Draft draft-ietf-httpbis-rfc6265bis.
- [6] Carl Boettiger. An introduction to Docker for reproducible research. *ACM SIGOPS Operating Systems Review*, 49(1):71–79, 2015.
- [7] John Brooke. SUS: A quick and dirty usability scale. In Patrick W. Jordan, Bruce Thomas, Bernard A. Weerdmeester, and Ian L. McClelland, editors, *Usability Evaluation in Industry*, pages 189–194. Taylor & Francis, 1996.
- [8] Thomas D. Cook and Donald T. Campbell. *Quasi-Experimentation: Design and Analysis Issues for Field Settings*. Houghton Mifflin, 1979.
- [9] Andy Georges, Dries Buytaert, and Lieven Eeckhout. Statistically rigorous Java performance evaluation. *ACM SIGPLAN Notices*, 42(10):57–76, 2007.
- [10] Jez Humble and David Farley. *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley, 2010.
- [11] Laura Inozemtseva and Reid Holmes. Coverage is not strongly correlated with test suite effectiveness. In *Proceedings of the 36th International Conference on Software Engineering (ICSE)*, pages 435–445, 2014.
- [12] Michael B. Jones, John Bradley, and Nat Sakimura. JSON web token (JWT). Technical Report RFC 7519, Internet Engineering Task Force, 2015.

- [13] Martin Kleppmann. *Designing Data-Intensive Applications*. O'Reilly Media, 2017.
- [14] Gerard Meszaros. *xUnit Test Patterns: Refactoring Test Code*. Addison-Wesley, 2007.
- [15] Todd Mytkowicz, Amer Diwan, Matthias Hauswirth, and Peter F. Sweeney. Producing wrong data without doing anything obviously wrong! In *Proceedings of ASPLOS XIV*, pages 265–276, 2009.
- [16] Sam Newman. *Building Microservices: Designing Fine-Grained Systems*. O'Reilly Media, 2 edition, 2021.
- [17] Jakob Nielsen. *Usability Engineering*. Academic Press, 1993.
- [18] Mark Nottingham and Erik Wilde. Problem details for HTTP APIs. Technical Report RFC 7807, Internet Engineering Task Force, 2016.
- [19] Mark Nottingham, Erik Wilde, and Sanjay Dalal. Problem details for HTTP APIs. Technical Report RFC 9457, Internet Engineering Task Force, 2023. Obsoleta la RFC 7807.
- [20] Michael Nygard. Documenting architecture decisions. Cognitect blog, 2011.
- [21] OWASP Foundation. OWASP top 10 — 2021, 2021.
- [22] OWASP Foundation. OWASP API security top 10 — 2023, 2023.
- [23] David L. Parnas. On the criteria to be used in decomposing systems into modules. *Communications of the ACM*, 15(12):1053–1058, 1972.
- [24] Niels Provos and David Mazières. A future-adaptable password scheme. In *Proceedings of the USENIX Annual Technical Conference*, pages 81–91, 1999.
- [25] Per Runeson and Martin Höst. Guidelines for conducting and reporting case study research in software engineering. *Empirical Software Engineering*, 14(2):131–164, 2009.
- [26] Geir Kjetil Sandve, Anton Nekrutenko, James Taylor, and Eivind Hovig. Ten simple rules for reproducible computational research. *PLoS Computational Biology*, 9(10):e1003285, 2013.
- [27] Mike West, Adam Barth, and Daniel Veditz. Content security policy level 3. W3C Working Draft, 2024.
- [28] Claes Wohlin, Per Runeson, Martin Höst, Magnus C. Ohlsson, Björn Regnell, and Anders Wesslén. *Experimentation in Software Engineering*. Springer, 2012.
- [29] Hong Zhu, Patrick A. V. Hall, and John H. R. May. Software unit test coverage and adequacy. *ACM Computing Surveys*, 29(4):366–427, 1997.